

Battery-Gated Adaptation for Battery-Constrained Quadruped Robots:

A Synthetic End-to-End Workflow Demonstration

Anonymous Authors

Abstract

Adaptation helps legged robots survive distribution shift, but adaptation itself consumes battery, latency, and onboard compute. This sample paper asks a deployment question: when battery is limited, should a quadruped adapt continuously, or should adaptation be treated as a resource-budgeted action? We instantiate that question on a synthetic benchmark that exists to exercise the full end-to-end workflow in this repository. The benchmark compares four policies: a static controller, periodic adaptation, always-on adaptation, and a battery-gated forward-only adaptation rule. In the low-battery terrain regime, the gated policy improves mission return from 0.67 to 0.75 relative to always-on adaptation while reducing adaptation-compute energy from 0.56 Wh to 0.13 Wh. In the low-battery payload regime, mission return improves from 0.69 to 0.77 with a similar reduction in compute cost. Averaged across the two low-battery regimes, the gated policy gains 0.08 mission return over always-on adaptation while using less than one third of its adaptation-compute energy. In high-battery regimes, always-on adaptation remains competitive, which points to a scheduling claim rather than a universal superiority claim. The main value of this artifact is twofold: it illustrates a credible battery-aware adaptation hypothesis, and it leaves a compiled paper that demonstrates the intended output shape of the repository’s fully automatic research pipeline.

1 Introduction

Legged robots already adapt well to terrain, payload, and contact variation when given enough sensing and online adaptation machinery (Kumar et al., 2021). A second line of work shows that energy-aware objectives can materially reshape locomotion behavior (Fu et al., 2021; Aboufazeli et al., 2023). Meanwhile, efficient test-time adaptation (TTA) has become credible on memory- and latency-constrained hardware (Hong et al., 2023; Jia et al., 2024; Xiao et al., 2026). What remains underexplored is the joint question: when a battery-constrained robot must spend energy both to move and to adapt, when is adaptation worth the cost?

This sample paper studies that question in the smallest honest setting available inside this repository. The repository does not ship a quadruped stack, a simulator, or a hardware benchmark. Instead, it ships the upstream workflow logic plus a tracked artifact tree. To make the full path concrete, we continue the earlier idea-discovery example with a synthetic benchmark stored in `results/demo_benchmark.json`. The benchmark is not presented as a robotics result. It is a workflow-complete demonstration of the hypothesis that adaptation should be scheduled as a resource-budgeted control action.

The synthetic benchmark compares four policies across terrain and payload shift under both low- and high-battery regimes: a static controller, periodic adaptation, always-on adaptation, and a battery-gated forward-only adaptation rule. The key pattern is a regime split. In low-battery regimes, the gated policy improves mission return over always-on adaptation while consuming far less adaptation-compute energy. In high-battery regimes, always-on adaptation remains competitive in raw return. The resulting claim is

deliberately narrow: the preferred deployment strategy depends on the regime, and explicit adaptation-cost accounting can change which policy is best.

This framing matters for two reasons. First, it is a sharper and more falsifiable question than “do TTA for quadrupeds.” Second, it matches the repository’s advertised end state: the pipeline should be able to leave behind not only idea notes but also a narrative handoff, a plan, and a final paper artifact. In that sense, the contribution of this sample is partly methodological. It shows what a complete local artifact trail can look like even before a real target project is wired in.

Our contributions are:

- a battery-aware adaptation scheduling hypothesis that emerges naturally from the earlier literature and idea-review stages;
- a small synthetic benchmark that makes the low-battery versus high-battery crossover explicit;
- a complete paper artifact tree, ending in a compiled PDF, that demonstrates the intended output of the repository’s one-command research workflow.

2 Related Work

Rapid quadruped adaptation. Rapid Motor Adaptation (RMA) established that a quadruped can learn a latent online adaptation mechanism that transfers from simulation to difficult real-world terrain (Kumar et al., 2021). That line of work makes online adaptation operationally credible, but it optimizes robustness first and does not explicitly ask whether adaptation itself should be budgeted when battery is limited.

Energy-aware locomotion. Energy-minimizing locomotion objectives can produce meaningful changes in gait structure and efficiency (Fu et al., 2021). Online stiffness and stride adaptation can also reduce cost of transport without requiring a large offline training set (Aboufazeli et al., 2023). These results make energy-aware adaptation plausible, but they price actuation cost rather than the internal cost of the adaptation mechanism.

Efficient test-time adaptation. Recent TTA work makes online adaptation cheaper in memory, compute, or latency. MECTA reduces the memory overhead of continual adaptation (Hong et al., 2023). TinyTTA uses early exits to preserve adaptation quality on edge devices (Jia et al., 2024). Progressive Embedding Alignment shows that backprop-free online adaptation can be competitive with only forward passes (Xiao et al., 2026). These papers motivate the forward-only primitive in our sample, but they are not about closed-loop locomotion or battery-aware scheduling.

The sample therefore sits at the intersection of these three threads. It does not claim a new quadruped controller or a new TTA algorithm. Its narrower claim is that once adaptation cost is modeled explicitly, the preferred adaptation schedule can change across deployment regimes.

3 Battery-Gated Forward-Only Adaptation

We consider a controller that can optionally invoke a lightweight forward-only adaptation module before acting. The module is motivated by backprop-free embedding alignment (Xiao et al., 2026): it updates a compact latent or embedding transform using only forward statistics, avoiding a full gradient-based inner loop.

At each timestep t , the scheduler estimates a robustness gain Δ_t^{robust} and an adaptation cost E_t^{adapt} . The battery state b_t determines how aggressively compute cost should be penalized. The scheduling score is

$$s_t = \Delta_t^{\text{robust}} - \lambda(b_t)E_t^{\text{adapt}}, \quad (1)$$

where $\lambda(b_t)$ increases as the remaining battery decreases. The controller adapts only when

$$a_t = \mathbb{I}[s_t > \tau], \quad (2)$$

with threshold τ tuned once for the benchmark.

This formulation is intentionally modest. It is not a new general TTA algorithm. Instead, it isolates a single scheduling decision: whether the controller should spend energy on adaptation right now. The four policies in the benchmark are then:

- **Static:** never adapt.
- **Periodic:** adapt on a fixed interval.
- **Always-on:** adapt at every step.
- **Battery-gated:** adapt only when the score s_t exceeds the threshold.

The value of this formulation is that it makes a falsifiable prediction. If adaptation is effectively free, always-on adaptation should dominate. If adaptation is expensive relative to its robustness gain, a gated policy should become preferable, especially in low-battery regimes.

4 Synthetic Benchmark

The repository does not include a locomotion codebase, a simulator, or battery instrumentation. To complete the end-to-end workflow honestly, we therefore instantiate a synthetic benchmark whose only purpose is to make the scheduling hypothesis concrete and to leave a realistic artifact tree. All values used in this section come from `results/demo_benchmark.json`.

The benchmark spans two shift families, terrain shift and payload shift, under two battery regimes: a low-battery state at 25% and a high-battery state at 80%. Each regime evaluates the four policies described above. For every policy we record mission return, distance traveled, cost of transport (CoT), falls per 100m, number of adaptation invocations, and adaptation-compute energy in watt-hours.

The benchmark is designed to test one qualitative crossover. Low-battery regimes should punish always-on adaptation when its compute burden becomes non-trivial. High-battery regimes may still reward aggressive adaptation, because the controller can afford to spend more battery to maintain robustness. We further include an adaptation-cost sweep and a low-battery ablation to show how the crossover depends on the explicit cost model.

5 Results

The main result is the low-battery crossover. In terrain shift at 25% battery, battery-gated adaptation reaches a mission return of 0.75, ahead of always-on adaptation at 0.67, while using only 0.13 Wh of adaptation-compute energy instead of 0.56 Wh. Payload shift shows the same pattern: 0.77 versus 0.69, with compute cost reduced from 0.49 Wh to 0.14 Wh. These are the regimes that support the paper’s core scheduling claim.

[AUTO_PLACEHOLDER] Controller schematic.

State history and battery state feed a small scheduler. The scheduler chooses between no update, periodic update, or forward-only embedding alignment.

The paper sample keeps this schematic as a placeholder because the repo does not ship a real quadruped controller implementation.

Figure 1: Controller schematic placeholder for the sample paper. The real repository pipeline can continue without manual artwork by leaving explicit placeholder figures in place.

Table 1: Synthetic mission-level benchmark used for the end-to-end workflow sample. Higher mission return and distance are better. Lower cost of transport, falls, and adaptation-compute energy are better.

Regime	Method	Return	Distance (m)	CoT	Falls / 100m	Compute Wh
Terrain shift, 25% battery	Static	0.54	184	1.31	0.33	0.00
	Periodic	0.64	227	1.18	0.23	0.18
	Always-on	0.67	241	1.24	0.20	0.56
	Battery-gated	0.75	276	1.08	0.13	0.13
Terrain shift, 80% battery	Static	0.63	232	1.25	0.30	0.00
	Periodic	0.76	285	1.13	0.18	0.20
	Always-on	0.83	319	1.07	0.11	0.56
	Battery-gated	0.81	311	1.05	0.10	0.21
Payload shift, 25% battery	Static	0.59	205	1.21	0.29	0.00
	Periodic	0.68	244	1.13	0.20	0.16
	Always-on	0.69	251	1.16	0.18	0.49
	Battery-gated	0.77	289	1.02	0.11	0.14
Payload shift, 80% battery	Static	0.66	241	1.16	0.24	0.00
	Periodic	0.78	292	1.07	0.16	0.18
	Always-on	0.84	321	1.01	0.09	0.49
	Battery-gated	0.83	316	0.99	0.09	0.20

High-battery regimes are deliberately less flattering to gating. Under terrain shift at 80% battery, always-on adaptation reaches 0.83 return, slightly ahead of battery-gated adaptation at 0.81. Payload shift is similarly close at 0.84 versus 0.83. This is useful rather than inconvenient: it prevents the paper from pretending that one schedule dominates everywhere. Averaged over the two low-battery regimes, the gated rule improves return by 0.08 while cutting adaptation-compute energy by roughly 0.39 Wh relative to always-on adaptation.

The adaptation-cost sweep sharpens the story. When the adaptation-cost scale is only 0.25, always-on adaptation remains competitive at 0.82 return, versus 0.80 for the gated policy. As cost rises to 2.0, always-on adaptation falls to 0.49 while the gated policy remains at 0.68. The crossover is therefore not an artifact of a single tuned operating point; it is exactly the kind of deployment-sensitive threshold that motivated the project.

The ablation indicates that the full gated rule benefits from both battery state and the compute-cost penalty. Removing battery-state awareness or omitting the cost term weakens both mission return and transport efficiency.

Table 2: Adaptation-cost sweep on the low-battery benchmark slice. The battery-gated policy degrades more slowly as adaptation cost increases.

Cost scale	Periodic	Always-on	Battery-gated
0.25	0.71	0.82	0.80
0.50	0.69	0.77	0.79
1.00	0.64	0.67	0.75
1.50	0.60	0.58	0.71
2.00	0.55	0.49	0.68

Table 3: Low-battery ablation on the terrain-shift slice. Removing battery state or compute-cost awareness weakens the policy.

Variant	Return	CoT
Full gated policy	0.75	1.08
No battery-state input	0.68	1.16
No compute-cost penalty	0.70	1.19
Slip-threshold heuristic	0.72	1.14

6 Discussion and Limitations

The benchmark supports a narrow but coherent claim: once adaptation has a non-negligible energy cost, the best adaptation schedule can depend on the deployment regime. That is the most useful scientific signal in the sample. It also aligns with the review pressure from the earlier workflow stages, which repeatedly pushed the project away from “invent a new TTA method” and toward “show a regime where adaptation scheduling matters.” The right reading of the result is therefore not “battery-gated adaptation is the new default,” but “battery-aware scheduling is a variable the deployment story must explain.”

At the same time, the limitations are substantial. The benchmark is synthetic. The numbers were created to continue the workflow honestly inside the repository, not to stand in for a real quadruped evaluation. The controller schematic and regime map remain placeholders because the repository does not contain an actual adaptation implementation to visualize. A real project would need at least three follow-on steps: instrument a real or simulated locomotion stack, validate the same crossover across multiple disturbance families, and replace the placeholder figures with real plots and system diagrams. Until then, the paper should be read as a workflow artifact with a plausible research hypothesis, not as scientific evidence about quadruped deployment.

These limitations are not incidental. They are the reason the paper is framed as an end-to-end workflow demonstration rather than a publishable robotics claim. The point of the sample is to prove artifact completeness and to make the final paper stage concrete in the repository itself.

7 Conclusion

This sample paper asked a deployment question: when a battery-constrained quadruped can spend energy both to move and to adapt, when should it adapt? On the synthetic benchmark used for this repository demonstration, low-battery regimes favor a battery-gated forward-only adaptation rule over always-on adaptation, while high-battery regimes keep always-on adaptation competitive. That pattern supports a scheduling interpretation rather than a universal superiority claim.

More importantly for the repository itself, the sample leaves behind the full artifact path that the work-

[AUTO_PLACEHOLDER] Regime map.
Low battery + strong shift: battery-gated policy wins.
High battery + strong shift: always-on remains competitive.
Low shift: static or periodic adaptation can be sufficient.

Figure 2: Regime summary placeholder. The sample keeps the figure explicit and replaceable rather than inventing a polished graphic without a real experiment backend.

flow is supposed to produce: early-stage research notes, an auto-review summary, a narrative handoff, a paper plan, LaTeX sources, and a compiled PDF. A real follow-on project can now swap the synthetic benchmark for a real locomotion stack without having to guess what the final artifact tree should look like, while preserving the same local file structure.

A Artifact Inventory

This appendix documents the local files that make the sample reproducible.

- `01_literature_review.md` through `04_research_review.md`: earlier-stage idea artifacts.
- `05_auto_review.md`: reviewer-driven refinement notes for the downstream paper story.
- `06_narrative_report.md`: paper-ready narrative handoff.
- `07_paper_plan.md`: structure and figure plan.
- `results/demo_benchmark.json`: synthetic benchmark data used in the paper.
- `results/render_demo_tables.py`: script that renders the paper tables and placeholders from the JSON artifact.

B Synthetic Benchmark Schema

Each benchmark regime stores the shift family, the battery state, and one row per method. Each method row contains mission return, distance, cost of transport, falls per 100m, adaptation calls, and adaptation-compute energy. The cost sweep and ablation tables are generated from the same JSON file.

C Build Command

The paper can be rebuilt locally from this folder with:

```
python3 results/render_demo_tables.py  
cd paper && latexmk -pdf main.tex
```

References

- Marjan Aboufazeli, Samare Filsoofi, Jonathan Gurney, Sandy Meek, and V. John Mathews. An online learning algorithm for adapting leg stiffness and stride angle for efficient quadruped robot trotting. *Frontiers in Robotics and AI*, 10, 2023. Article 1139528.
- Zipeng Fu, Ashish Kumar, Jitendra Malik, and Deepak Pathak. Minimizing energy consumption leads to the emergence of gaits in legged robots. In *Proceedings of the Conference on Robot Learning*, 2021. OpenReview id: PfC1Jr6gvuP.
- Junyuan Hong, Lingjuan Lyu, Jiayu Zhou, and Michael Spranger. Mecta: Memory-economic continual test-time model adaptation. In *International Conference on Learning Representations*, 2023.
- Hong Jia, Young Kwon, Alessio Orsino, Ting Dang, Domenico Talia, and Cecilia Mascolo. Tinytta: Efficient test-time adaptation via early-exit ensembles on edge devices. In *Advances in Neural Information Processing Systems*, 2024.
- Ashish Kumar, Zipeng Fu, Deepak Pathak, and Jitendra Malik. Rma: Rapid motor adaptation for legged robots. In *Proceedings of Robotics: Science and Systems*, 2021. doi: 10.15607/RSS.2021.XVII.011.
- Ma Xiao, Young D. Kwon, Pan Zhou, and Dong Ma. Architecture-agnostic test-time adaptation via backprop-free embedding alignment. In *International Conference on Learning Representations*, 2026.